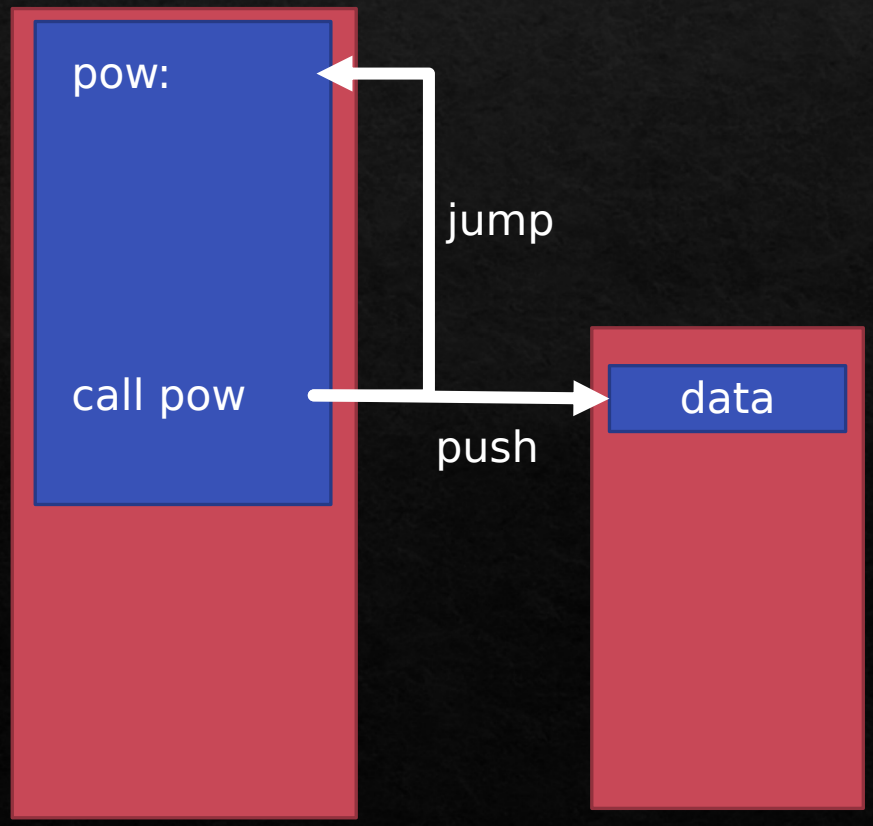
Рекурсия

понятие о рекурсии, примеры использования рекурсии, рекурсия и итерации, хвостовая рекурсия

Будем называть «рекурсивной» такую функцию, которая может быть выражена в терминах самой себя. Например функция возведения в натуральную степень:

$$\begin{cases} pow(a,1) = a \\ pow(a,n) = a \cdot pow(a,n-1) \end{cases}$$

Механизм функций C++ позволят реализовать подобные функции путём вызова функции по названию.




```
17  double integral_power(double base, unsigned exp) {
18      if (1 == exp ) return base;
19      return base * integral_power(base, exp-1);
20  }
```

Анатомия рекурсивно заданной функции

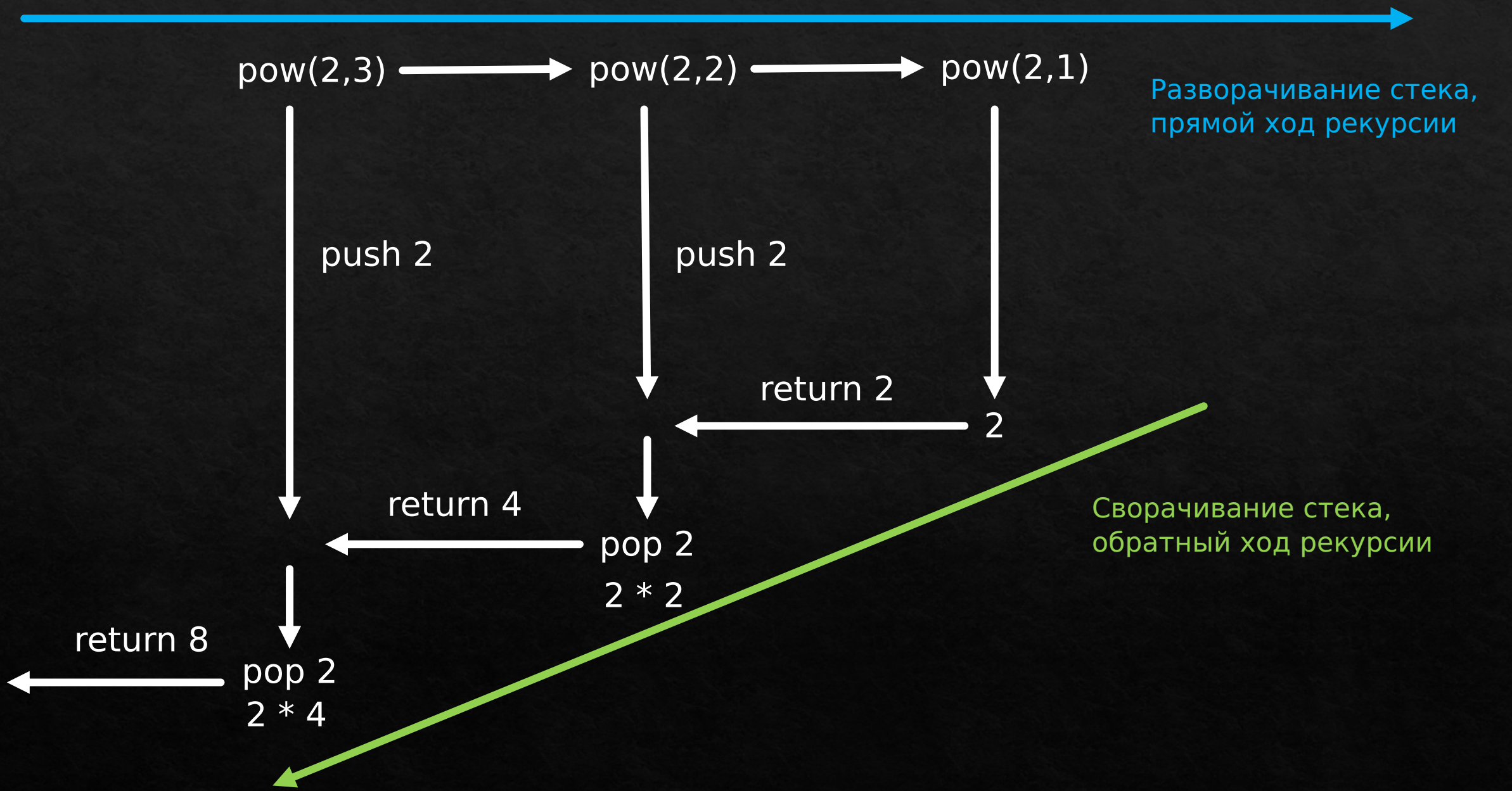
```
17  double integral_power(double base, unsigned exp) {
18      if (1 == exp ) return base;
19      return base * integral_power(base, exp-1);
20  }
```

База рекурсии – ситуация, позволяющая вычислить значение не прибегая к рекурсивному вызову.

Некоторое выражение, включающее в себя рекурсивный вызов.

Непосредственно рекурсивный вызов.

«Разворачивание» и «сворачивание» стека вызовов



Примеры

Факториал

$$\begin{cases} f(1) = 1 \\ f(n) = n \cdot f(n-1) \end{cases}$$

```
17  unsigned long long factorial(unsigned long long n) {
18      if (1 == n) return 1;
19      return n * factorial(n - 1);
20  }
```

Наибольший общий делитель

$$\begin{cases} gcd(a,0) = a \\ gcd(a,b), b > a = gcd(b,a) \\ gcd(a,b), a \leq b = gcd(b, a \bmod b) \end{cases}$$

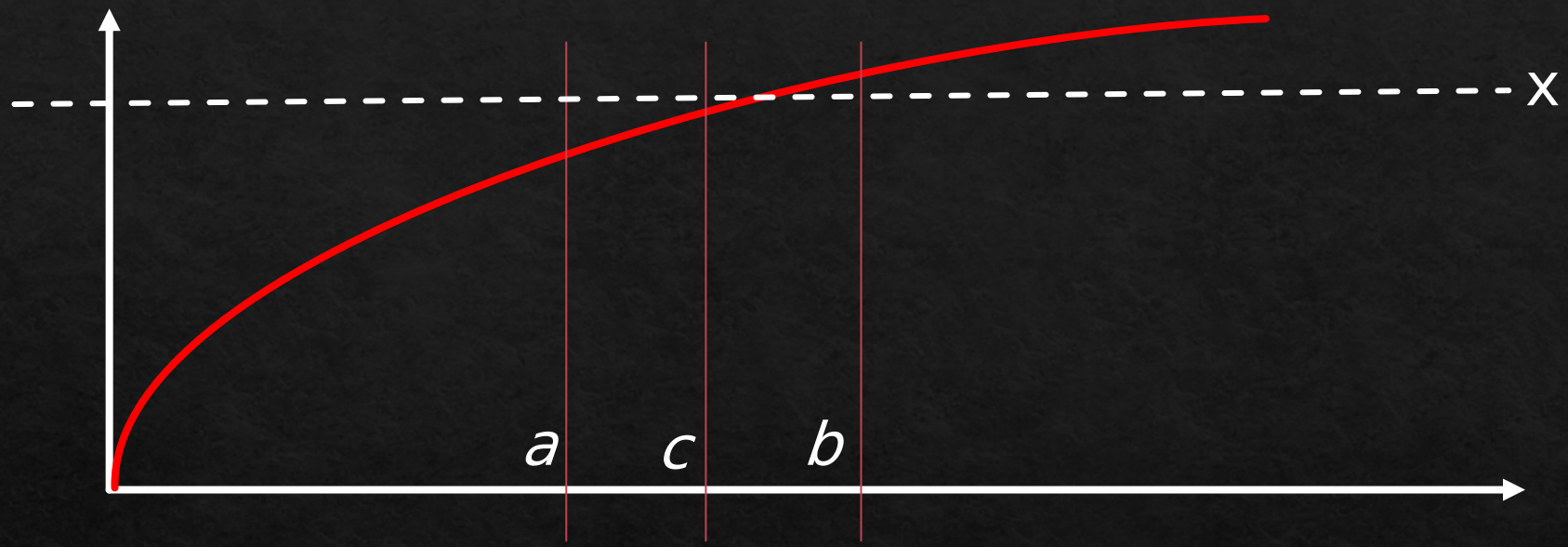
```
27  unsigned gcd(unsigned a, unsigned b) {
28      if (0 == b) return a;
29      if (b > a) return gcd(b, a);
30      return gcd(b, a % b);
31  }
```

Сумма элементов массива, индукция

Пусть мы умеем решать задачу для N элементов и знаем способы получить решение: при добавлении одного элемента к набору и ровно для одного элемента. Тогда мы можем написать решение индуктивно через рекурсивный вызов.

```
33  int inductive_array_sum(int arr[], unsigned l_idx, unsigned r_idx) {
34      if (1 == r_idx - l_idx) return arr[l_idx];
35      return arr[l_idx] + inductive_array_sum(arr,l_idx + 1, r_idx);
36  }
```

Поиск квадратного корня методом дихотомии




```
45  double dichotomy_sqrt(double x, double a, double b, double tolerance) {
46      auto center = (a + b) / 2.;
47      if (b - a < tolerance) return center;
48      if (center * center > x) return dichotomy_sqrt(x, a, center, tolerance);
49      return dichotomy_sqrt(x, center, b, tolerance);
50  }
```

Максимум в массиве методом разделения

Max on the left | Max on the right

```
37  int max_arr(int arr[], unsigned l_idx, unsigned r_idx) {
38      if (1 == r_idx - l_idx) return arr[l_idx];
39      auto center_idx = l_idx + (r_idx - l_idx) / 2;
40      int left_max = max_arr(l_idx, center_idx);
41      int right_max = max_arr(center_idx, r_idx);
42      return max2(left_max, right_max);
43  }
```

```
33  int max2(int a, int b) {
34      return a > b ? a : b;
35  }
```

Рекурсия как альтернатива итерациям

Печать строки заданное количество раз

```
45  void print_count(unsigned counter) {
46      if (0 == counter)
47          return;
48      cout << "Hello recursion!" << '\n';
49      print_count(counter - 1);
50  }
```

```
52  void print_count_loop(unsigned counter) {
53      while (0 != counter) {
54          cout << "Hello loop!" << '\n';
55          ++counter;
56      }
57  }
```

Сумма цифр числа

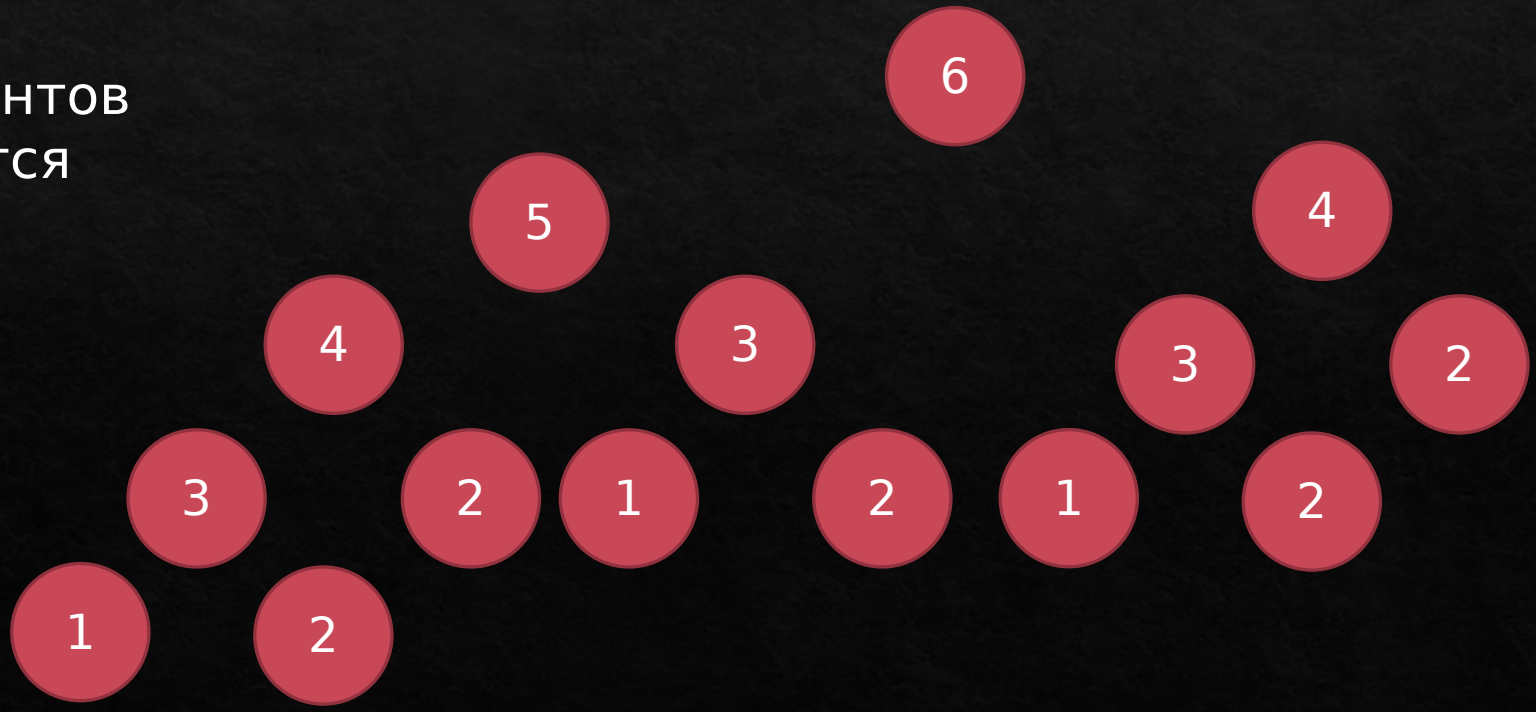
```
59  unsigned digit_sum_rec(unsigned remnant) {
60      if (!remnant) return 0;
61      return remant%10 + digit_sum_rec(remnant/10);
62  }
```

```
64  unsigned digit_sum_loop(unsigned a) {
65      auto remnant = a;
66      unsigned sum = 0;
67      while(remnant) {
68          auto digit = remnant%10;
69          remnant /= 10;
70      }
71      return sum;
72  }
```

Неудачная рекурсия...

```
1   unsigned long long naive_fibonacci_rec(unsigned long long N) {
2       if (1 == N) return 1;
3       if (2 == N) return 1;
4       return naive_fibonacci_rec(N - 1) + naive_fibonacci_rec(N - 2);
5   }
```

Вычисление многих элементов повторяется, хотя требуется только два: последний и предпоследний.



Фибоначчи в цикле

```
 7  unsigned long long fibonacci_loop(unsigned long long N) {
 8      auto prev = 1ull, curr = 1ull; // first and second
 9      while (1 != N) {
10          auto tmp = curr;
11          curr += prev;
12          prev = tmp;
13          --N
14      }
15      return prev;
16  }
```

Хвостовая рекурсия

```
18  unsigned long long fibonacci_tail_rec(
19      unsigned long long prev, // 1 at call
20      unsigned long long curr, // 1 at call
21      unsigned long long N) {
22      if (1 == N) return prev;
23      return fibonacci_tail_rec(curr, curr + prev, N - 1);
24  }
```

```
 7  unsigned long long fibonacci_loop(unsigned long long N) {
 8      auto prev = 1ull, curr = 1ull; // first and second
 9      while (1 != N) {
10          auto tmp = curr;
11          curr += prev;
12          prev = tmp;
13          --N
14      }
15      return prev;
16  }
```

Удобный вызов

```
26  unsigned long long fibonacci_tail_rec_adapter(unsigned long long N) {
27      return fibonacci_tail_rec(1ull, 1ull, N);
28  }
```

```
18  unsigned long long fibonacci_tail_rec(
19      unsigned long long prev, // 1 at call
20      unsigned long long curr, // 1 at call
21      unsigned long long N) {
22      if (1 == N) return prev;
23      return fibonacci_tail_rec(curr, curr + prev, N - 1);
24  }
```

```
30  double tail_rec_integral_power(double res, double base, double exp) {
31      if (1 == exp) return base;
32      return tail_rec_integral_power(res * base, base, exp - 1);
33  }
```

```
17   double integral_power(double base, unsigned exp) {
18       if (1 == exp ) return base;
19       return base * integral_power(base, exp-1);
20   }
```

```
35  unsigned long long tail_rec_factorial(unsigned long long res, unsigned long long n) {
36      if (1 == n) return 1;
37      return tail_rec_factorial(res * n, n - 1);
38  }
```

```
17  unsigned long long factorial(unsigned long long n) {
18      if (1 == n) return 1;
19      return n * factorial(n - 1);
20  }
```

```
27  unsigned gcd(unsigned a, unsigned b) {
28      if (0 == b) return a;
29      if (b > a) return gcd(b, a);
30      return gcd(b, a % b);
31  }
```